EXPERIMENT -6

AIM- Create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling. Implement triggers for salary validation and audit logging. Test edge cases.

OBJECTIVES

1. To understand the concept of stored procedures in SQL.
2. To create a stored procedure for transferring an employee between departments.
3. To implement input validation and error handling using SIGNAL.
4. To understand SQL triggers and their execution timing.
5. To create a trigger for validating employee salaries.
6. To create an audit table for maintaining salary-change history.
7. To create an audit trigger for recording salary modifications.
8. To test valid and invalid inputs using different edge cases.

THEORY

Stored Procedure

A stored procedure is a named collection of SQL statements stored in the database and executed when called.

Stored procedures are useful for:

- Reusing frequently performed operations.
- Encapsulating business logic.
- Performing validation.
- Handling transactions.

Parameterized Stored Procedure

A stored procedure can accept parameters from the user or application.

For example: transfer_employee(emp_id, new_dept_id)

Here:

- `emp_id` identifies the employee.

- `new_dept_id` identifies the destination department.

Error Handling

Error handling allows a procedure to respond appropriately when an invalid condition occurs.

For example, an error can be generated when:

- Employee does not exist.
- Department does not exist.
- Employee is already in the requested department.
- Invalid input is supplied.

Transaction

A transaction groups multiple database operations into a single logical unit.

The main transaction commands are:

- START TRANSACTION
- COMMIT
- ROLLBACK

Trigger

A trigger is a database object that automatically executes when a specified event occurs on a table.

Triggers can execute:

- BEFORE INSERT
- AFTER INSERT
- BEFORE UPDATE
- AFTER UPDATE
- BEFORE DELETE
- AFTER DELETE

Salary Validation Trigger

A salary validation trigger can ensure that invalid salary values are not inserted or updated.

For example, a salary can be required to be greater than zero.

A BEFORE INSERT trigger validates newly inserted employees, while a BEFORE UPDATE trigger validates salary changes.

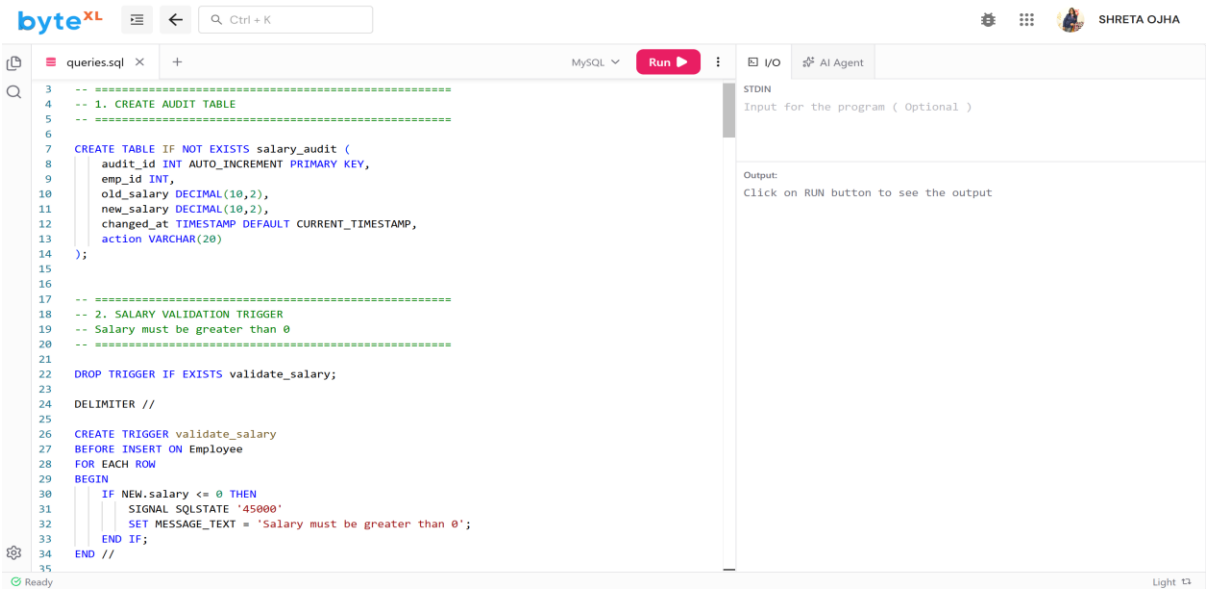
Audit Logging

Audit logging records important changes made to database records.

For salary changes, an audit table can store:

- Employee ID
- Old salary
- New salary
- Date and time of change
- Type of operation

SOURCE CODE




```
-- ======================================================
-- 1. CREATE AUDIT TABLE
-- ======================================================

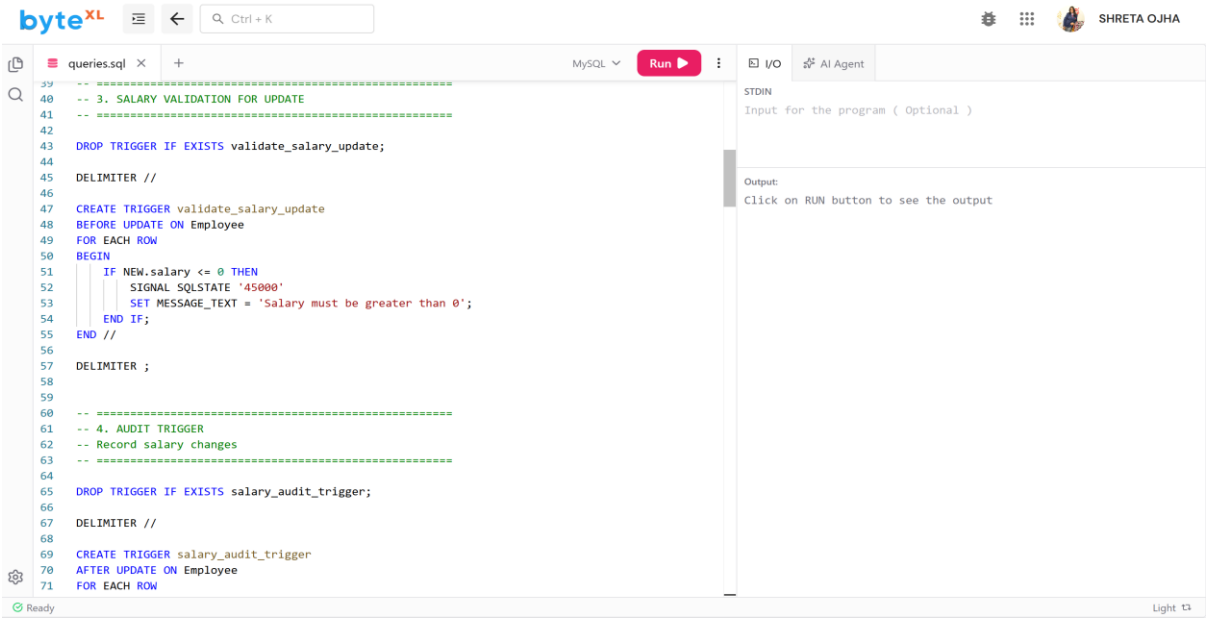
CREATE TABLE IF NOT EXISTS salary_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    action VARCHAR(20)
);


-- ======================================================
-- 2. SALARY VALIDATION TRIGGER
-- Salary must be greater than 0
-- ======================================================

DROP TRIGGER IF EXISTS validate_salary;

DELIMITER //

CREATE TRIGGER validate_salary
BEFORE INSERT ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than 0';
    END IF;
END //
```




```
-- ======================================================
-- 3. SALARY VALIDATION FOR UPDATE
-- ======================================================

DROP TRIGGER IF EXISTS validate_salary_update;

DELIMITER //

CREATE TRIGGER validate_salary_update
BEFORE UPDATE ON Employee
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than 0';
    END IF;
END //

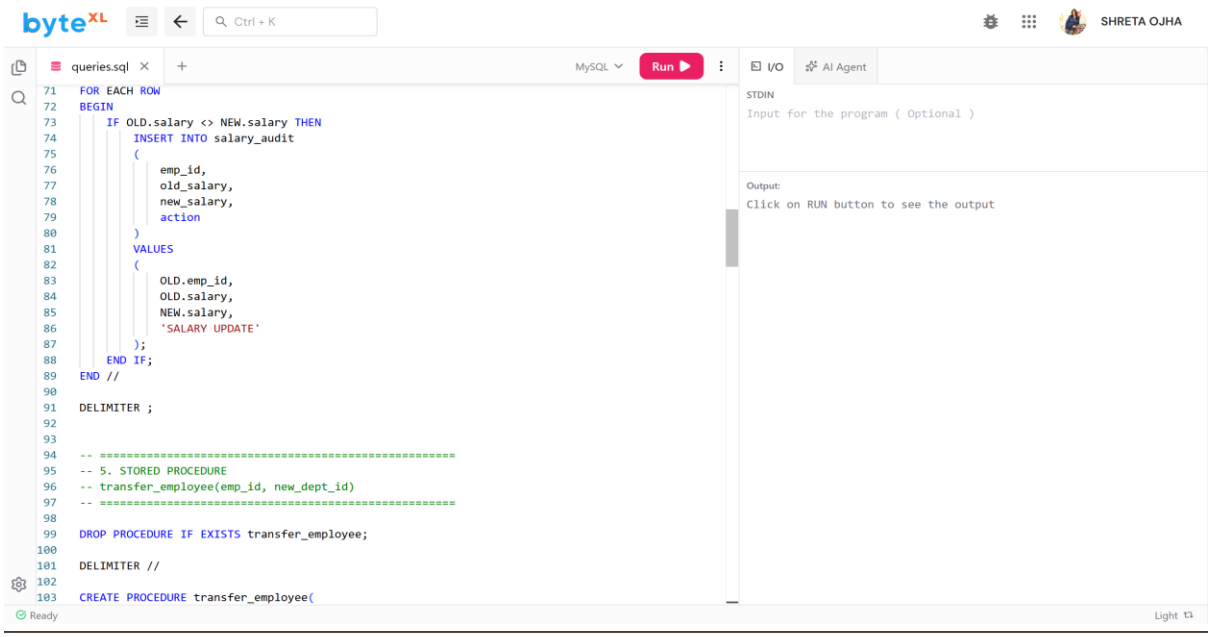
DELIMITER ;

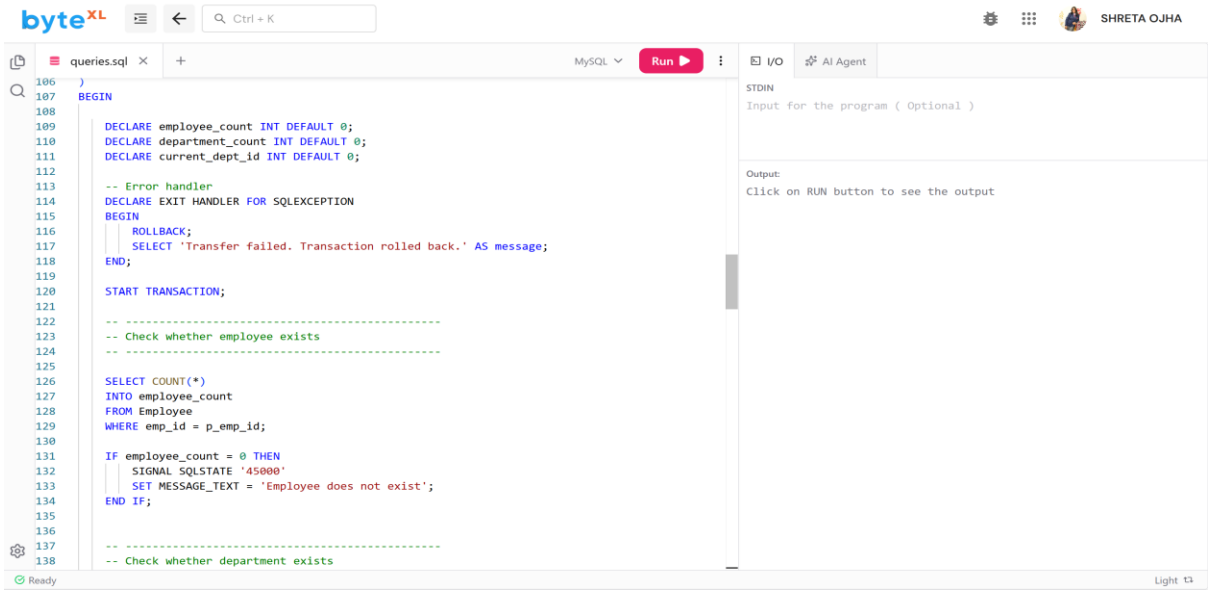

-- ======================================================
-- 4. AUDIT TRIGGER
-- Record salary changes
-- ======================================================

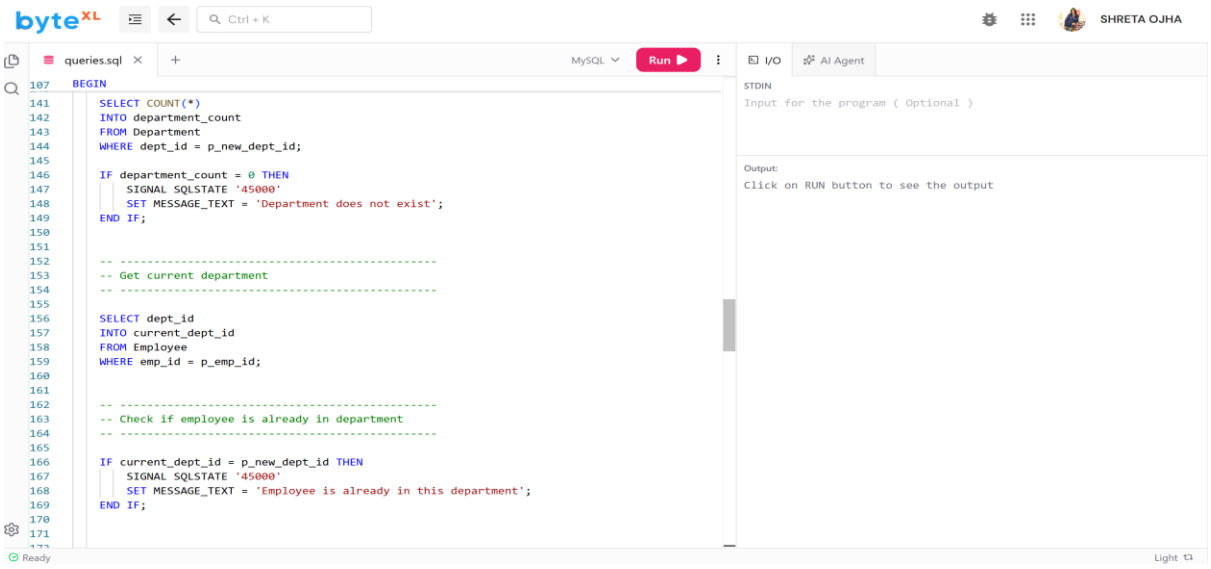
DROP TRIGGER IF EXISTS salary_audit_trigger;

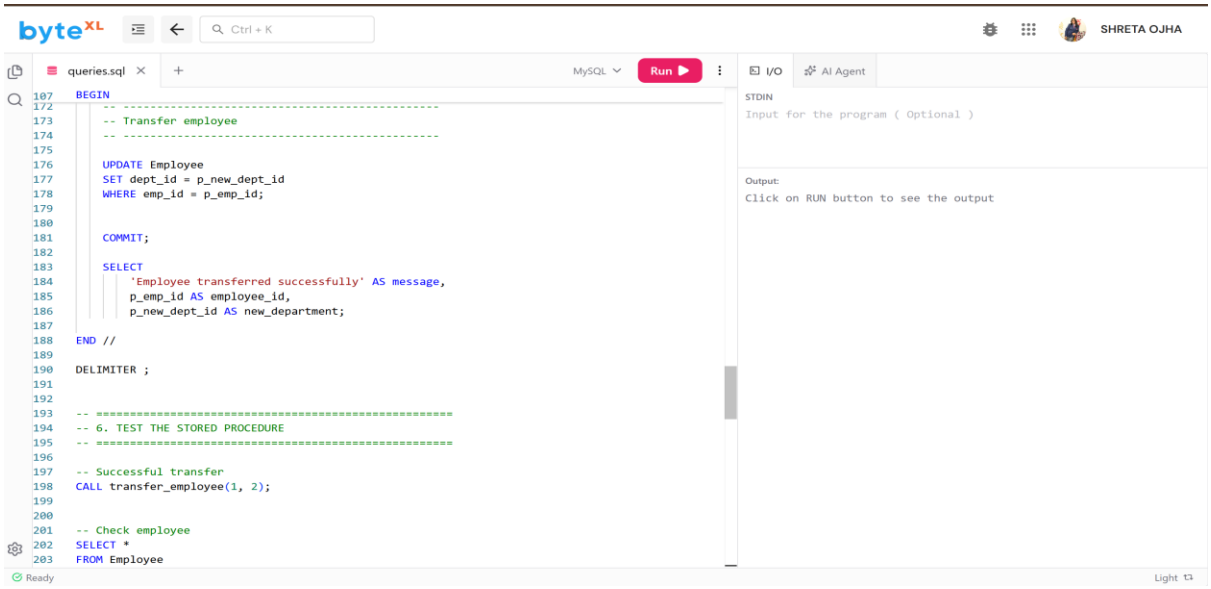
DELIMITER //

CREATE TRIGGER salary_audit_trigger
AFTER UPDATE ON Employee
FOR EACH ROW
```


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
FOR EACH ROW
BEGIN
IF OLD.salary <> NEW.salary THEN
INSERT INTO salary_audit
(
emp_id,
old_salary,
new_salary,
action
)
VALUES
(
OLD.emp_id,
OLD.salary,
NEW.salary,
'SALARY UPDATE'
);
END IF;
END //
DELIMITER ;
-- ==
-- 5. STORED PROCEDURE
-- transfer_employee(emp_id, new_dept_id)
-- ==
DROP PROCEDURE IF EXISTS transfer_employee;
DELIMITER //
CREATE PROCEDURE transfer_employee(
Ready
Light


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
)
BEGIN
DECLARE employee_count INT DEFAULT 0;
DECLARE department_count INT DEFAULT 0;
DECLARE current_dept_id INT DEFAULT 0;
-- Error handler
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
ROLLBACK;
SELECT 'Transfer failed. Transaction rolled back.' AS message;
END;
START TRANSACTION;
-- --
-- Check whether employee exists
-- --
SELECT COUNT(*)
INTO employee_count
FROM Employee
WHERE emp_id = p_emp_id;
IF employee_count = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Employee does not exist';
END IF;
-- --
-- Check whether department exists
Ready
Light


byteXL
Ctrl + K
SHRETA OJHA
queries.sql
MySQL
Run
I/O
AI Agent
STDIN
Input for the program (Optional)
Output:
Click on RUN button to see the output
BEGIN
SELECT COUNT(*)
INTO department_count
FROM Department
WHERE dept_id = p_new_dept_id;
IF department_count = 0 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Department does not exist';
END IF;
-- --
-- Get current department
-- --
SELECT dept_id
INTO current_dept_id
FROM Employee
WHERE emp_id = p_emp_id;
-- --
-- Check if employee is already in department
-- --
IF current_dept_id = p_new_dept_id THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Employee is already in this department';
END IF;
Ready
Light




```
BEGIN
    -- ------------------------------------------------
    -- Transfer employee
    -- ------------------------------------------------

    UPDATE Employee
    SET dept_id = p_new_dept_id
    WHERE emp_id = p_emp_id;


    COMMIT;

    SELECT
        'Employee transferred successfully' AS message,
        p_emp_id AS employee_id,
        p_new_dept_id AS new_department;

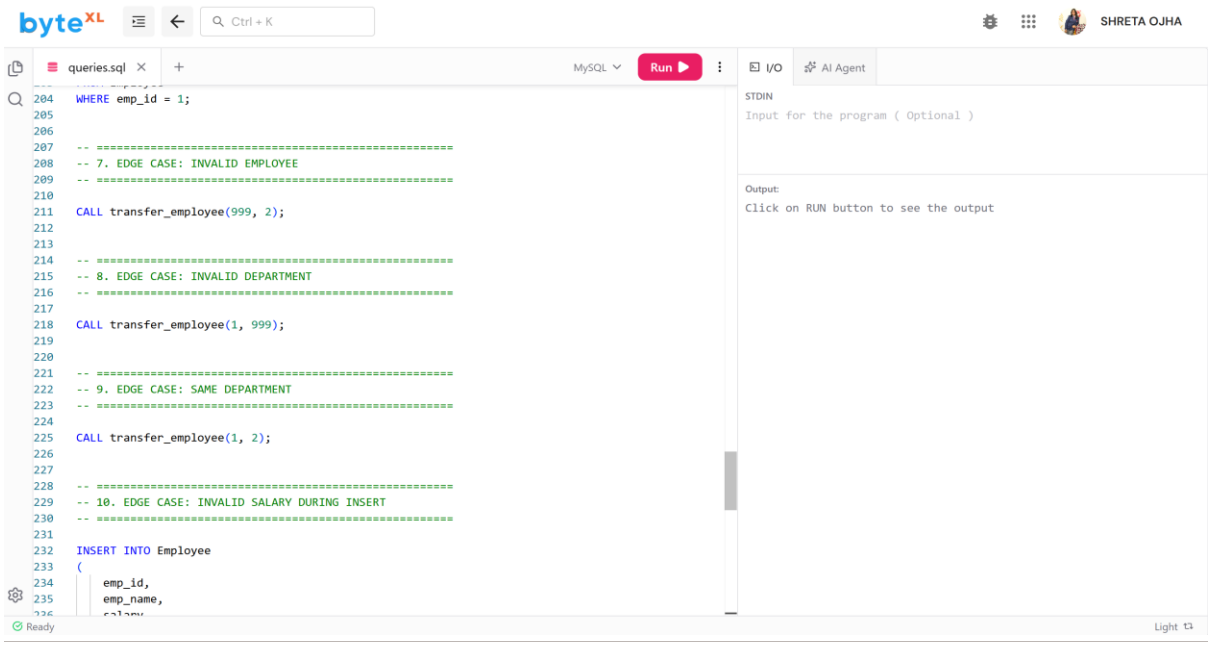
END //

DELIMITER ;


-- ============================================================
-- 6. TEST THE STORED PROCEDURE
-- ============================================================

-- Successful transfer
CALL transfer_employee(1, 2);


-- Check employee
SELECT *
FROM Employee
```




```
WHERE emp_id = 1;


-- ============================================================
-- 7. EDGE CASE: INVALID EMPLOYEE
-- ============================================================

CALL transfer_employee(999, 2);


-- ============================================================
-- 8. EDGE CASE: INVALID DEPARTMENT
-- ============================================================

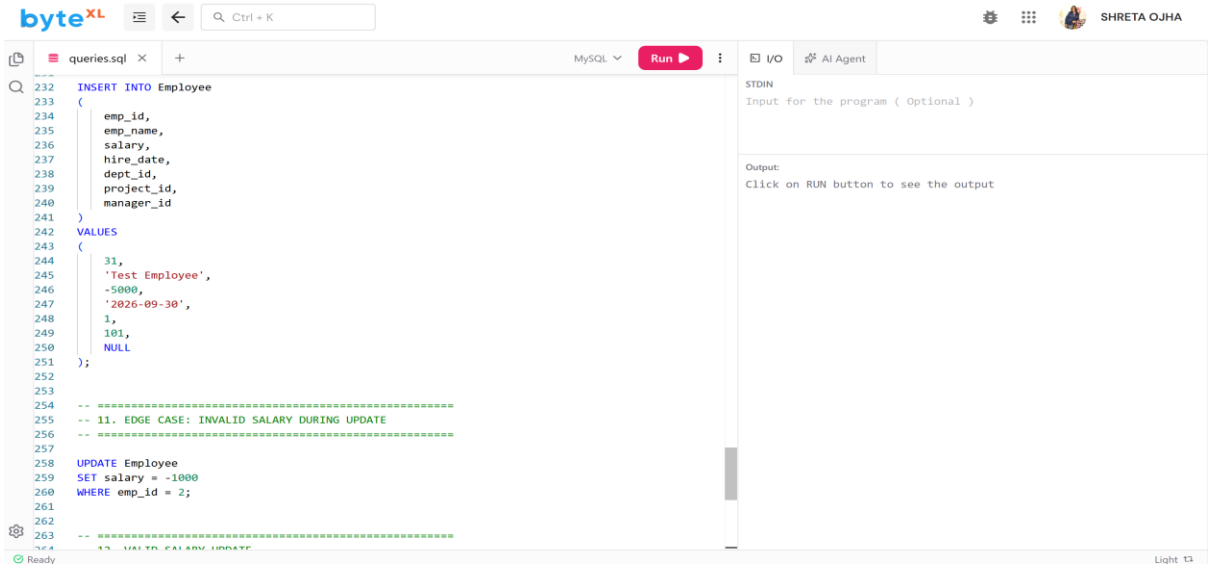
CALL transfer_employee(1, 999);


-- ============================================================
-- 9. EDGE CASE: SAME DEPARTMENT
-- ============================================================

CALL transfer_employee(1, 2);


-- ============================================================
-- 10. EDGE CASE: INVALID SALARY DURING INSERT
-- ============================================================

INSERT INTO Employee
(
    emp_id,
    emp_name,
```




```
INSERT INTO Employee
(
    emp_id,
    emp_name,
    salary,
    hire_date,
    dept_id,
    project_id,
    manager_id
)
VALUES
(
    31,
    'Test Employee',
    -5000,
    '2026-09-30',
    1,
    101,
    NULL
);


-- ============================================================
-- 11. EDGE CASE: INVALID SALARY DURING UPDATE
-- ============================================================

UPDATE Employee
SET salary = -1000
WHERE emp_id = 2;


-- ============================================================
```

```sql
-- ====================================================
-- 12. VALID SALARY UPDATE
-- This should succeed and create an audit record
-- ====================================================

UPDATE Employee
SET salary = 90000
WHERE emp_id = 2;


-- ====================================================
-- 13. CHECK AUDIT LOG
-- ====================================================

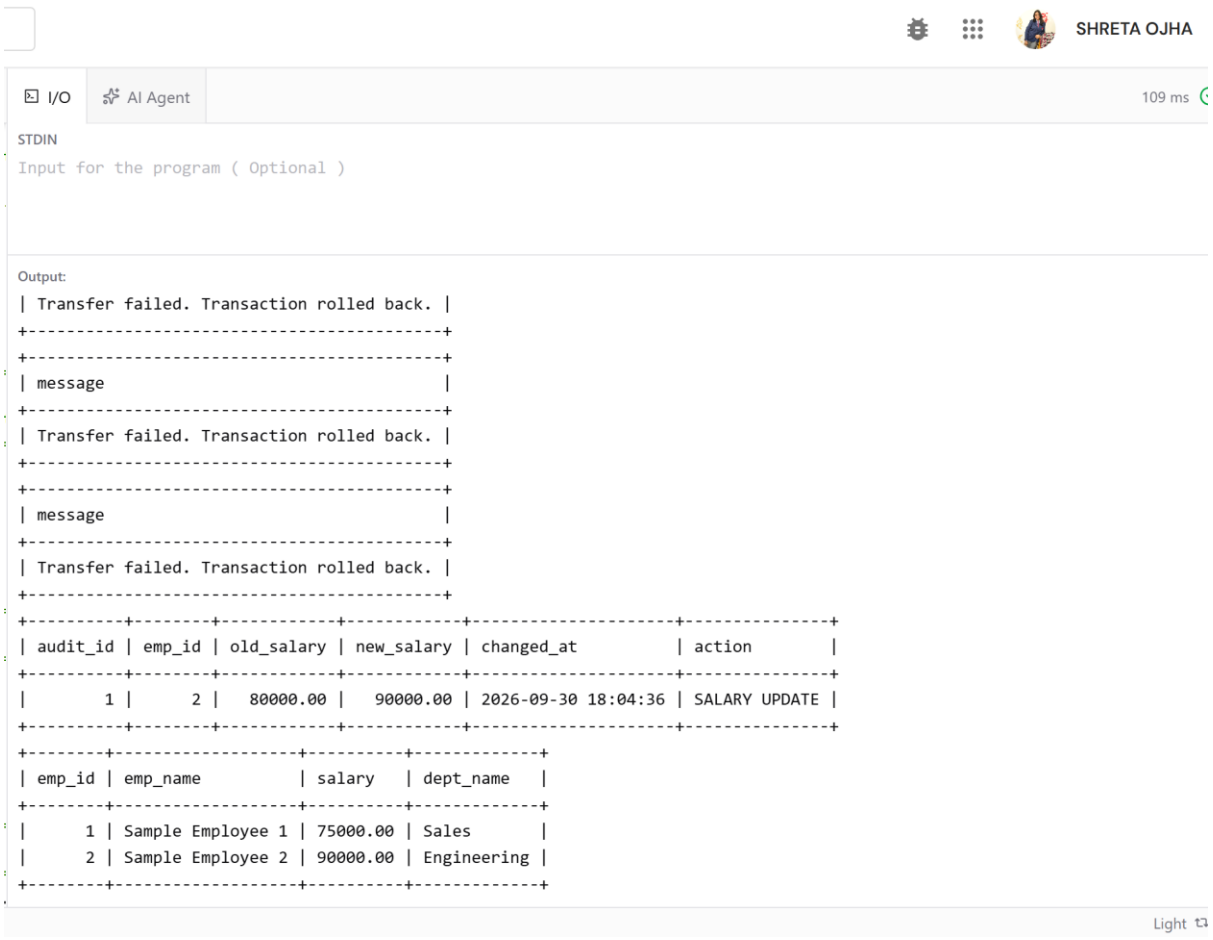
SELECT *
FROM salary_audit
ORDER BY changed_at DESC;


-- ====================================================
-- 14. VERIFY EMPLOYEE DATA
-- ====================================================

SELECT
    e.emp_id,
    e.emp_name,
    e.salary,
    d.dept_name
FROM Employee e
JOIN Department d
ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

OUTPUT

```
+----------------------------------+-------------+----------------+
| message                          | employee_id | new_department |
+----------------------------------+-------------+----------------+
| Employee transferred successfully |           1 |              2 |
+----------------------------------+-------------+----------------+
+--------+--------------------+----------+------------+---------+------------+------------+
| emp_id | emp_name           | salary   | hire_date  | dept_id | project_id | manager_id |
+--------+--------------------+----------+------------+---------+------------+------------+
|      1 | Sample Employee 1 | 75000.00 | 2024-01-15 |       2 |        101 |       NULL |
+--------+--------------------+----------+------------+---------+------------+------------+
+----------------------------------------+
| message                                |
+----------------------------------------+
| Transfer failed. Transaction rolled back. |
+----------------------------------------+
+----------------------------------------+
| message                                |
+----------------------------------------+
| Transfer failed. Transaction rolled back. |
+----------------------------------------+
+----------------------------------------+
| message                                |
+----------------------------------------+
```



OBSERVATIONS/OUTCOME

1. The transfer_employee stored procedure was successfully created with input validation and error handling.
2. Valid employee transfers between departments were successfully performed.
3. Invalid employee and department IDs were rejected by the procedure.
4. Transfers to the employee's existing department were prevented.
5. The salary validation triggers successfully rejected zero, negative, and NULL salary values.
6. Valid salary modifications were successfully allowed.
7. The salary audit trigger automatically recorded salary changes in the SalaryAudit table.
8. Edge cases were tested to verify the reliability of the procedure and triggers.